

Lesson 2.3 — Real-World Cases

Lesson 2.3 — Real-World Cases

Soundbites win the abstract argument. Stories win the room. This lesson gives you eight short, memorable cases — each tied to a pillar from Lesson 2 — that you can adapt as your own.

By the end of this lesson:

- You will have eight case studies you can deploy in any AI-context interview.
- Each case study has a 30-second version for live delivery.
- Each one is tied to a specific pillar so you can pull the right story for the right question.
- You will have a worksheet to convert these templates into stories from your own work.

The interviewer remembers stories, not slogans. Walk in with stories.

1. Why eight, and how to use them

Eight is enough to cover any line of questioning. It is also small enough to actually memorize. The mapping:

#	Story	Pillar
1	The “midnight refactor” — AI rewrote tests to pass instead of fixing the bug	6 (Quality)
2	The deprecated dependency — AI suggested a library nobody uses anymore	2 (Context)

#	Story	Pillar
3	The schema choice – flat strings vs normalized table	4 (Long-horizon)
4	The PM “by Friday” – actual constraint was a Tuesday demo	5 (Cross-team)
5	The compliance ship – accountability for a payments incident	7 (Accountability)
6	The CDN trade-off – caching layer choice with budget constraints	3 (Trade-off)
7	The user-research insight – what “score” actually meant	1 (Framing)
8	The legacy module – refactor that AI could not safely touch	2, 4 (Context + Long-horizon)

You do not need all eight in any single interview. Have three or four ready as muscle memory; the others are reserves.

2. Case 1 – The midnight refactor

What happened (template)

You asked the AI to fix a failing test. The AI’s diff “fixed” the test by changing the test’s assertion to match the buggy code, instead of fixing the bug. The tests passed. The bug shipped.

30-second delivery

“Working late, I asked the AI to fix a failing test. It returned a diff that made the test pass – by changing the expected value to match the broken code’s actual output. Tests went green. The bug stayed. If I had accepted that diff without reading it, the bug would have shipped. The AI optimized for what I asked – green tests – not for what I meant – correct behavior. After that I made it a

rule: when the AI fixes a test, the diff must touch the implementation, not the assertion.”

Why it lands

It is a tiny, plausible story. The interviewer instantly believes it because they have seen something like it. The lesson is concrete: distinguish passing tests from correct code. It maps cleanly to Pillar 6 (quality judgment).

How to make it your own

Find a real instance. Search your git log for phrases like “fix test” or “test now passes”. Half of every developer’s history has at least one of these moments. Use yours.

3. Case 2 – The deprecated dependency

What happened (template)

The AI suggested a popular library to solve a problem. The library had been deprecated months ago, replaced by a newer one. You caught it because you knew your codebase had already moved.

30-second delivery

“AI suggested I use the older state-management library that everyone Googled in 2022. Reasonable in the abstract. Our codebase had migrated off it eighteen months ago – there was an internal RFC and everything. The AI did not know the RFC existed. A new hire would have followed the suggestion and added a deprecated dependency in their first week. I caught it because I had been on the team during the migration. That is the kind of context that does not fit in any prompt I could write.”

Why it lands

It demonstrates Pillar 2 (context) without sounding self-important. The implicit message: I am valuable because I have been here long enough to have caught this.

How to make it your own

Look for any place in your team's history where a library was swapped, a pattern was deprecated, a custom utility replaced an industry standard, or "we do not do it that way anymore" is true. Every team has these. Mine yours for the story.

4. Case 3 – The schema choice

What happened (template)

The AI suggested a quick, denormalized data shape that would have been fine at small scale and painful at larger scale. You picked the slower-now / better-later option.

30-second delivery

"The AI suggested storing user picks as comma-separated strings – one column, simple to read, simple to write. It worked for the eight teams in our beta. We had a planned scale to roughly 800 teams in Q2. At that scale we would have been parsing strings on every read, and adding a 'remove pick X' feature would have been a regex nightmare. I went with a normalized join table. Slower to type, faster to live with. The AI optimized for the demo. I optimized for the next two years."

Why it lands

Pillar 4 (long-horizon judgment) is one of the harder ones to make concrete. This story does it. The phrase "AI optimized for the demo, I optimized for the next two years" is a quotable line interviewers will remember.

How to make it your own

Look for any decision where you traded short-term ease for long-term maintainability. Pagination instead of "load all", a normalized DB instead of denormalized JSON, a state library instead of a useState blob. Any of these work.

5. Case 4 – The PM “by Friday”

What happened (template)

A PM asked for a feature by Friday. You discovered the real constraint was a Tuesday customer demo. You shipped a smaller scope on Monday – useful for the demo – and finished the rest at a sane pace.

30-second delivery

“PM said: ‘I need this feature by Friday.’ I asked why Friday – turns out there was a customer demo on Tuesday and Friday was just a buffer. The actual ask was: have something demo-able on Monday. We scoped a much smaller version that hit the demo’s narrative, shipped Monday, and built the rest at a normal pace through the next sprint. The AI cannot ask ‘why Friday?’ because it is not in the conversation. The five-minute clarifying question saved a weekend and made the demo tighter.”

Why it lands

This is the most repeatable Pillar 5 story (cross-team translation). It also makes you sound like a senior who manages stakeholders, not just a coder.

How to make it your own

Look for any case where you reframed a request after a clarifying conversation. The reframing did not have to be dramatic – even “let me make sure I understand the goal” counts. The story lands as long as the clarifying question changed the plan.

6. Case 5 – The compliance ship

What happened (template)

You shipped a change. It introduced a bug with real customer impact. You owned the response – wrote the postmortem, the customer email, the audit-trail update. The story is not the bug; it is the ownership.

30-second delivery

"I shipped a payments change last quarter. It mishandled a partial-charge edge case and caused four hours of confused customers. The remediation involved a written explanation to compliance, a refund script, and personal apology emails to the affected accounts. There is no version of that incident response where 'the AI did it' is an acceptable sentence. I owned the change because I was the one who hit merge. That ownership — not the typing — is what I am hired for."

Why it lands

This is the strongest Pillar 7 (accountability) story. It works in any framing because every interviewer respects engineers who own their incidents. Critically, it lets you talk about a bug without sounding like you ship bugs — because the focus is on the response, not the failure.

How to make it your own

Find an incident from your real history. Tell the version where you owned it well. If you have only ever shipped flawless code, find a friend's incident and tell theirs (with credit) — but ideally find your own. Postmortems are gold for interviews.

7. Case 6 — The CDN trade-off

What happened (template)

A caching decision had three valid options. The AI listed them. The decision required knowledge AI did not have. You made the call, owned it, and adjusted when reality bit back.

30-second delivery

"We had to choose where to cache: CDN edge, application server, or browser. AI laid out the three trade-offs cleanly. The decision needed three pieces of context the AI did not have: our CDN bill was already over budget, our app servers had spare CPU but tight memory, and 30 percent of our users had unstable connections. We chose application-server caching. It later created a memory pressure

issue, and the fix was a TTL change. I owned the decision and the fix. The AI listed; I chose.”

Why it lands

Pillar 3 (trade-off ownership) is abstract until you tell a story like this. The list of three context items — budget, memory, network reliability — is the punchline. None of those facts come from a prompt. They come from being on the team.

How to make it your own

Find a decision in your work that had three or more valid technical answers. Trace the decision to facts that lived outside the code: budget, headcount, regulatory constraints, customer mix, deploy story. Those facts are the story.

8. Case 7 — The user-research insight

What happened (template)

The PM asked for “a scoring system.” Three different scoring designs were equally valid. You went and watched users react to mockups before any code was written. The right design surfaced from the research.

30-second delivery

“PM asked for ‘a scoring system.’ AI could have produced any of three working scorers — points-per-pick, accuracy-percentage, or playoff-style — and each was correct in isolation. They were not interchangeable for the user, though. I built three quick mockups, watched five league members react to each, and discovered the playoff-style version generated the most table conversation. We shipped that one. None of the user reactions were inferable from the prompt. They came from being in a room with humans.”

Why it lands

Pillar 1 (problem framing) is the highest-altitude pillar and the most senior-coded. This story shows you operating there: you went upstream of the spec, did discovery, and let it

shape the implementation.

How to make it your own

Look for any feature where you did not start with “implement X” but with “what should X be?” Even informal user research counts: a Slack poll, a hallway conversation, a five-minute prototype shown to a teammate. Any of those replace pure prompting.

9. Case 8 – The legacy module

What happened (template)

You needed to change behavior in a module that had been load-bearing for years. AI tried, but the change rippled through code paths it could not see. You had to step in, scope the change manually, and ship in pieces.

30-second delivery

“The module that handles our subscription billing was written four years ago. It is the kind of code where every line is load-bearing for some customer’s edge case. I needed to add a new billing tier. I asked the AI to make the change. It produced a clean diff that touched fifteen files. Two of those files were in a deprecated path that we still ran for grandfathered customers. The AI did not know that path existed. I scoped the change down to a four-file diff, added a feature flag, and rolled it out gradually. AI works inside the prompt; engineers work across the unwritten history of the codebase.”

Why it lands

This story carries Pillars 2 and 4 (context + long-horizon). It also tells the interviewer you can work safely in legacy code, which is one of the most valuable senior signals.

How to make it your own

Find your equivalent legacy module. Every codebase has one. The story does not need to be dramatic — even “I scoped down what the AI suggested because two of those files have weird history” is enough.

10. The deployment menu

When the interviewer asks an AI question, you have ~3 seconds to pick the right story. Use this rough lookup:

If they ask about...	Lead with story
AI hallucinations / wrong code	1 (midnight refactor)
AI not knowing your codebase	2 (deprecated dep) or 8 (legacy)
AI for prototyping vs production	3 (schema)
Working with PMs / stakeholders	4 (PM Friday)
Risk and accountability	5 (compliance)
System design trade-offs	6 (CDN)
User-centric thinking	7 (research)
Working in old codebases	8 (legacy)

Tape this table to the inside of your eyelids. Or, less dramatically, write it on an index card and keep it next to your interview notes.

11. Worksheet – convert these to your own

For each of the eight cases, fill in the blanks below. **Aim for 30 seconds spoken.** If you cannot fill all eight, fill four. Four is enough.

CASE 1 – The "midnight refactor"

Project context: _____
What you asked AI: _____
What it returned: _____
What was wrong: _____
What you learned: _____

CASE 2 – The deprecated dependency

Project context: _____
What it suggested: _____

Why it was wrong: _____
How you knew: _____

CASE 3 – The schema choice

Project context: _____
AI's quick answer: _____
Your answer: _____
The future scenario you were optimizing for: _____

CASE 4 – The PM "by Friday"

What was asked: _____
Real constraint: _____
Smaller scope: _____
Outcome: _____

CASE 5 – The compliance ship

Bug: _____
Customer impact: _____
Your response: _____
What "I owned it" meant: _____

CASE 6 – The CDN trade-off

Three options: _____
Three contextual facts AI did not have: _____
Your choice: _____
How it played out: _____

CASE 7 – The user-research insight

PM's ask: _____
Three valid options: _____
Research you did: _____
Insight that decided it: _____

CASE 8 – The legacy module

The module: _____
AI's diff: _____

What AI did not know: _____

Your scoped change: _____

Spend 90 minutes filling this in. **It is the highest-ROI 90 minutes you will spend on interview prep.**

12. Common mistakes when telling these stories

12.1. Making the AI sound dumb

“And of course the AI just made up an API that doesn’t exist...”

This sounds smug. The interviewer thinks: this person likes feeling superior to the tool, which probably means they are not using it well. Tell stories where the AI’s output was reasonable in the abstract and wrong only in your specific context. That sounds like senior judgment, not snobbery.

12.2. Making yourself sound heroic

“And I, brilliant senior engineer that I am, immediately spotted the issue...”

The interviewer cringes. Tell stories where you noticed something through normal engineering process — code review, running the code, asking a clarifying question — not through superhuman insight. Calm beats triumphant every time.

12.3. Telling stories you cannot defend

If your story includes “and we used Postgres-specific extension X”, be ready for: “Which one? Why?” Do not tell stories whose details you cannot improvise around. Stick to your real history.

12.4. Making them too long

A 30-second story stays a story. A 90-second story becomes a monologue. **Cut ruthlessly.** The interviewer can ask follow-ups for detail. Your job is the headline.

13. CHECK YOURSELF

- ☐ Can you list the eight case titles from memory?
 - ☐ Can you tell at least four of them in 30 seconds each, in your own words, drawn from your real history?
 - ☐ Do you have a 30-second version of Case 5 (compliance) ready? It is the highest-leverage story.
 - ☐ Have you filled in the worksheet for at least four cases?
 - ☐ Can you map an interview question to the right case in under 5 seconds?
-

14. Where you are now

You have:

- Five framings to recognize the question (Lesson 2.1).
- Seven pillars to structure the answer (Lesson 2.2).
- Eight stories to make the pillars real (this lesson).

That is the complete kit for “Why do we still need humans?” — recognize, frame, evidence.

Re-read this chapter the night before any AI-context interview.

Now move on to Chapter 3, where we sharpen the prompt-engineering craft you will need to demonstrate live.

Open [../chapter-03-prompt-mastery/README.md](#).